

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – PENCARIAN NILAI MAX MIN



NAMA : MOH. CHANDRA WARDANA

NIM : 109082530025

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pencarian nilai maksimum dan minimum merupakan salah satu konsep dasar dalam algoritma dan pemrograman yang digunakan untuk menemukan nilai terbesar atau terkecil dalam sekumpulan data. Proses ini biasanya dilakukan pada struktur data seperti array, di mana setiap elemen akan dibandingkan satu per satu secara berurutan (sekuensial) hingga ditemukan nilai ekstrem yang diinginkan. Konsep ini sangat penting karena sering digunakan dalam berbagai kasus nyata seperti menentukan nilai tertinggi siswa, harga terendah barang, atau performa terbaik suatu sistem.

Algoritma pencarian nilai maksimum atau minimum bekerja dengan cara menginisialisasi nilai awal sebagai nilai ekstrem sementara, biasanya diambil dari elemen pertama array. Selanjutnya, algoritma akan melakukan iterasi terhadap seluruh elemen lainnya dan membandingkannya dengan nilai ekstrem sementara tersebut. Jika ditemukan nilai yang lebih besar (untuk maksimum) atau lebih kecil (untuk minimum), maka nilai ekstrem akan diperbarui. Proses ini terus dilakukan hingga seluruh data selesai diperiksa.

Dalam implementasinya menggunakan bahasa pemrograman seperti Go, pencarian nilai maksimum dan minimum dapat dilakukan dengan menggunakan perulangan seperti `for`. Selain itu, konsep ini juga dapat diterapkan pada tipe data terstruktur seperti `struct`, sehingga tidak hanya nilai saja yang didapatkan, tetapi juga informasi terkait data tersebut. Dengan demikian, pemahaman algoritma ini menjadi dasar penting dalam pengolahan data dan pengambilan keputusan dalam pemrograman.

B. Guided

1. MinMax_Array

```

1 package main
2
3 import "fmt"
4
5 func nilaiTerbesar(array []int) (int, int) {
6     var idx int
7     var nilaiTerbesar int = array[0]
8     var idxDitemukan int = 0
9     for idx = 1; idx < len(array); idx++ {
10         if array[idx] > nilaiTerbesar {
11             nilaiTerbesar = array[idx]
12             idxDitemukan = idx
13         }
14     }
15     return nilaiTerbesar, idxDitemukan
16 }
17
18 func nilaiTerkecil(array []int) (int, int) {
19     var idx int
20     var nilaiTerkecil int = array[0]
21     var idxDitemukan int = 0
22     for idx = 1; idx < len(array); idx++ {
23         if array[idx] < nilaiTerkecil {
24             nilaiTerkecil = array[idx]
25             idxDitemukan = idx
26         }
27     }
28     return nilaiTerkecil, idxDitemukan
29 }
30
31 func main() {
32     var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}
33
34     var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
35     var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])
36
37     fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d", nilaiMin, idxNilaiMin)
38     fmt.Println()
39     fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d", nilaiMaks, idxNilaiMaks)
40
41 }
42

```

Penjelasan Code :

Program ini dirancang untuk mencari nilai ekstrem, yaitu nilai terbesar dan nilai terkecil, beserta posisi indeksinya di dalam sebuah array bilangan bulat (*integer*). Di dalamnya terdapat dua fungsi utama, `nilaiTerbesar` dan `nilaiTerkecil`, yang keduanya bekerja dengan cara menginisialisasi variabel penampung menggunakan elemen pertama array dan kemudian melakukan iterasi melalui seluruh elemen menggunakan perulangan `for` untuk membandingkan nilai saat ini dengan nilai yang sudah disimpan. Jika ditemukan angka yang lebih besar pada fungsi pertama atau lebih kecil pada fungsi kedua, maka nilai serta indeksinya akan diperbarui, hingga akhirnya fungsi mengembalikan dua nilai sekaligus (*multiple return values*) untuk ditampilkan di fungsi utama.

Output :

```

PS D:\109082530025_MOH-Chandra-Wardana> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 9 - Modul 10\Guided\guided1\guided1.go"
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2

```

Penjelasan Output :

Output dari program ini akan menampilkan dua baris informasi berdasarkan data statis array [45, 23, 98, 66, 12]. Pertama, program mencetak nilai terkecil yaitu 12 yang ditemukan pada indeks ke-4. Kedua, program mencetak nilai terbesar yaitu 98 yang ditemukan pada indeks ke-2. Hasil ini diperoleh setelah fungsi membandingkan semua angka di dalam array tersebut secara berurutan.

2. MinMax_Struct

```
1 package main
2
3 import "fmt"
4
5 type mahasiswa struct {
6     nama string
7     nim string
8     ipk float64
9 }
10
11 type arrMahasiswa []mahasiswa
12
13 func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
14     var mhsTerbesar mahasiswa = array[0]
15     var idDitemukan int = 0
16     var i int
17
18     for i = 1; i < len(array); i++ {
19         if array[i].ipk > mhsTerbesar.ipk {
20             mhsTerbesar = array[i]
21             idDitemukan = i
22         }
23     }
24
25     return mhsTerbesar, idDitemukan
26 }
27
28 func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
29     var mhsTerkecil mahasiswa = array[0]
30     var idDitemukan int = 0
31     var i int
32
33     for i = 1; i < len(array); i++ {
34         if array[i].ipk < mhsTerkecil.ipk {
35             mhsTerkecil = array[i]
36             idDitemukan = i
37         }
38     }
39
40     return mhsTerkecil, idDitemukan
41 }
42
43 func main() {
44     var arrMhs arrMahasiswa
45     var i, j int
46     for i = 0; i < len(arrMhs); i++ {
47         fmt.Println("DAFTAR MAHASISWA INDEKS KE-", i)
48         fmt.Println("Masukkan nama : ")
49         mhs := &arrMhs[i]
50         fmt.Println("Masukkan NIM : ")
51         mhs.nim = fmt.Sprintf("%s", arrMhs[i].nim)
52         fmt.Println("Masukkan IPK : ")
53         mhs.ipk = fmt.Sprintf("%f", arrMhs[i].ipk)
54         fmt.Println("-----")
55     }
56
57     for j = 0; j < len(arrMhs); j++ {
58         fmt.Println("DAFTAR MAHASISWA INDEKS KE-", j)
59         mhs := &arrMhs[j]
60         fmt.Println("Nama : ", mhs.nama)
61         fmt.Println("NIM : ", mhs.nim)
62         fmt.Println("IPK : ", mhs.ipk)
63         fmt.Println("-----")
64     }
65
66     var mhsMaks, mhsMin mahasiswa
67     var idMaks, idMin int
68
69     mhsMaks, idMaks = IPKTerbesar(arrMhs[:])
70     mhsMin, idMin = IPKTerkecil(arrMhs[:])
71
72     fmt.Println("---- DAFTAR IPK TERKECIL & TERBESAR MAHASISWA ----")
73     fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMaks.ipk, mhsMaks.nama, mhsMaks.nim, idMaks)
74     fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMin.ipk, mhsMin.nama, mhsMin.nim, idMin)
75 }
76
```

Penjelasan Code :

Program ini merupakan implementasi pengelolaan data terstruktur menggunakan struct untuk menyimpan informasi mahasiswa yang terdiri dari nama, NIM, dan IPK, yang kemudian dikelola dalam sebuah array bertipe data bentukan tersebut. Program memiliki fungsi IPKTerbesar dan IPKTerkecil yang menerima parameter berupa *slice* dari *struct* mahasiswa untuk melakukan pencarian data berdasarkan nilai IPK paling tinggi dan paling rendah. Di dalam fungsi main, pengguna diminta untuk memasukkan

data secara interaktif melalui input terminal, kemudian program akan mengiterasi data tersebut untuk ditampilkan kembali dan memanggil fungsi pencarian guna menentukan mahasiswa mana yang memiliki prestasi IPK tertinggi serta terendah beserta rincian identitas dan lokasi indeksinya di dalam array.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 9 - Modul 10\Guided\guided2\guided2.go"
DATA MAHASISWA INDEKS KE- 0
Masukkan nama : chandra
Masukkan NIM :109082530025
Masukkan IPK : 4
=====
DATA MAHASISWA INDEKS KE- 1
Masukkan nama : dana
Masukkan NIM :10982530021
Masukkan IPK : 3
=====
```

Penjelasan Output :

Output yang dihasilkan akan diawali dengan ringkasan seluruh data mahasiswa yang telah diinputkan oleh pengguna, mencakup nama, NIM, dan IPK masing-masing. Setelah daftar tersebut ditampilkan, program akan mencetak hasil analisis berupa informasi detail mahasiswa dengan IPK terbesar dan terkecil, di mana output tersebut mencantumkan nilai IPK-nya (dengan format dua angka di belakang koma), nama mahasiswa yang bersangkutan, nomor NIM mereka, serta pada indeks keberapa data tersebut tersimpan di dalam sistem.

C. Unguided

1. Soal Unguided 1

Program Pencarian Berat Kelinci Minimum dan Maksimum

Jawaban :

A screenshot of a code editor with a dark background and light-colored text. The code is written in Go and is enclosed in a light gray border. The code is as follows:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var n int
7     var berat [1000]float64
8     var min, max float64
9
10    fmt.Print("Masukkan jumlah kelinci: ")
11    fmt.Scan(&n)
12
13    for i := 0; i < n; i++ {
14        fmt.Printf("Masukkan berat kelinci ke-%d: ", i+1)
15        fmt.Scan(&berat[i])
16    }
17
18    min = berat[0]
19    max = berat[0]
20
21    for i := 1; i < n; i++ {
22        if berat[i] < min {
23            min = berat[i]
24        }
25        if berat[i] > max {
26            max = berat[i]
27        }
28    }
29
30    fmt.Println("Berat terkecil:", min)
31    fmt.Println("Berat terbesar:", max)
32 }
```

Penjelasan Program :

Program ini digunakan untuk mencari berat terkecil dan terbesar dari sejumlah data berat kelinci yang dimasukkan oleh pengguna. Pertama, program meminta input jumlah data, kemudian menyimpan setiap berat ke dalam array. Nilai awal minimum dan maksimum diambil dari elemen pertama, lalu program melakukan perulangan untuk membandingkan setiap elemen berikutnya. Jika ditemukan nilai yang lebih kecil dari minimum atau lebih besar dari maksimum, maka nilai tersebut akan diperbarui hingga seluruh data selesai diperiksa.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 9 - Modul 10\Unguided\unguided1\unguided1.go"
Masukkan jumlah kelinci: 10
Masukkan berat kelinci ke-1: 2.5
Masukkan berat kelinci ke-2: 3.8
Masukkan berat kelinci ke-3: 1.1
Masukkan berat kelinci ke-4: 1.8
Masukkan berat kelinci ke-5: 5.5
Masukkan berat kelinci ke-6: 4.6
Masukkan berat kelinci ke-7: 8.9
Masukkan berat kelinci ke-8: 10.9
Masukkan berat kelinci ke-9: 1.5
Masukkan berat kelinci ke-10: 9.7
Berat terkecil: 1.1
Berat terbesar: 10.9
```

Penjelasan output:

Output tersebut menunjukkan hasil dari proses pencarian nilai minimum dan maksimum berdasarkan data yang dimasukkan pengguna, di mana program berhasil mengidentifikasi bahwa berat terkecil adalah 1.1 dan berat terbesar adalah 10.9 setelah seluruh data dibandingkan secara berurutan.

2. Soal Unguided 2

Program Pengelompokan Berat Ikan per Wadah

Jawaban :

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var x, y int
7     var ikan [1000]float64
8
9     fmt.Print("Masukkan jumlah ikan: ")
10    fmt.Scan(&x)
11
12    fmt.Print("Masukkan kapasitas per wadah: ")
13    fmt.Scan(&y)
14
15    for i := 0; i < x; i++ {
16        fmt.Printf("Berat ikan ke-%d: ", i+1)
17        fmt.Scan(&ikan[i])
18    }
19
20    jumlahWadah := (x + y - 1) / y
21    var total float64
22
23    fmt.Println("Total berat per wadah:")
24    for i := 0; i < jumlahWadah; i++ {
25        var sum float64
26        for j := i * y; j < (i+1)*y && j < x; j++ {
27            sum += ikan[j]
28        }
29        fmt.Println(sum)
30        total += sum
31    }
32
33    rata := total / float64(jumlahWadah)
34    fmt.Println("Rata-rata berat per wadah:", rata)
35 }
```

Penjelasan Program :

Program ini bertujuan untuk menghitung total berat ikan dalam setiap wadah berdasarkan kapasitas tertentu, serta menghitung rata-rata berat per wadah. Program membaca jumlah ikan dan kapasitas tiap wadah, kemudian mengelompokkan data berat ikan ke dalam beberapa wadah menggunakan perulangan bersarang. Setiap kelompok dijumlahkan dan disimpan sebagai total berat per wadah, kemudian dihitung rata-ratanya dari seluruh wadah.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 9 - Modul 10\Unguided\unguided2\unguided2.go"
Masukkan jumlah ikan: 5
Masukkan kapasitas per wadah: 3
Berat ikan ke-1: 1
Berat ikan ke-2: 2
Berat ikan ke-3: 3
Berat ikan ke-4: 4
Berat ikan ke-5: 5
Total berat per wadah:
6
9
Rata-rata berat per wadah: 7.5
```

Penjelasan output :

Output tersebut menunjukkan bahwa data ikan berhasil dikelompokkan ke dalam beberapa wadah sesuai kapasitas, di mana setiap wadah menampilkan total berat ikan di dalamnya, kemudian program menghitung rata-rata dari total berat seluruh wadah sehingga diperoleh nilai rata-rata sebesar 5.

3. Soal Unguided 3

Program Data Berat Balita (Min, Max, Rata-rata)

Jawaban :

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      var n int
7      var berat [100]float64
8      var min, max, total float64
9
10     fmt.Print("Masukkan jumlah balita: ")
11     fmt.Scan(&n)
12
13     for i := 0; i < n; i++ {
14         fmt.Printf("Masukkan berat balita ke-%d: ", i+1)
15         fmt.Scan(&berat[i])
16     }
17
18     min = berat[0]
19     max = berat[0]
20
21     for i := 0; i < n; i++ {
22         if berat[i] < min {
23             min = berat[i]
24         }
25         if berat[i] > max {
26             max = berat[i]
27         }
28         total += berat[i]
29     }
30
31     rata := total / float64(n)
32
33     fmt.Println("Berat minimum:", min)
34     fmt.Println("Berat maksimum:", max)
35     fmt.Println("Rata-rata:", rata)
36 }

```

Penjelasan Program :

Program ini digunakan untuk mengolah data berat balita dengan mencari nilai minimum, maksimum, dan rata-rata. Program menerima input jumlah data dan nilai berat, kemudian menggunakan perulangan untuk membandingkan setiap data guna menentukan nilai ekstrem serta menjumlahkan seluruh data untuk perhitungan rata-rata.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 9 - Modul 10\Unguided\unguided3\tempCodeRunnerFile.go"
Masukkan jumlah balita: 5
Masukkan berat balita ke-1: 4.7
Masukkan berat balita ke-2: 3.4
Masukkan berat balita ke-3: 5.5
Masukkan berat balita ke-4: 2.1
Masukkan berat balita ke-5: 10
Berat minimum: 2.1
Berat maksimum: 10
Rata-rata: 5.14
```

Penjelasan output :

Output tersebut menunjukkan hasil pengolahan data berat balita di mana program berhasil menentukan nilai terkecil, terbesar, serta menghitung rata-rata dari seluruh data yang dimasukkan sehingga memberikan gambaran statistik sederhana dari data tersebut.

D. Kesimpulan

Kesimpulan dari praktikum ini adalah bahwa pencarian nilai maksimum dan minimum merupakan teknik dasar yang sangat penting dalam pengolahan data menggunakan array, di mana prosesnya dilakukan dengan membandingkan setiap elemen secara berurutan untuk menemukan nilai ekstrem yang diinginkan. Melalui implementasi menggunakan bahasa Go, dapat dipahami bahwa algoritma ini tidak hanya sederhana tetapi juga efisien untuk berbagai kasus, baik pada data bertipe dasar maupun terstruktur, serta dapat dikembangkan untuk menghitung nilai lain seperti rata-rata. Praktikum ini juga memberikan pemahaman bahwa logika perulangan dan kondisi menjadi kunci utama dalam menyelesaikan permasalahan pencarian data, sehingga dapat diterapkan dalam berbagai kebutuhan pemrograman di dunia nyata.

